

RESEARCH ARTICLE

Detecting Domain Names Generated by DGAs With Low False Positives in Chinese Domain Names

HUIJU LEE¹, JEONG DO YOO¹, SEONGHOON JEONG², (Member, IEEE),
AND HUY KANG KIM¹, (Member, IEEE)

¹School of Cybersecurity, Korea University, Seoul 02841, Republic of Korea

²Division of Artificial Intelligence Engineering, Sookmyung Women's University, Seoul 04310, Republic of Korea

Corresponding author: Huy Kang Kim (cenda@korea.ac.kr)

This work was supported by Korea Agency for Infrastructure Technology Advancement (KAIA) through the Ministry of Land, Infrastructure, and Transport under Grant RS-2021-KA162334.

ABSTRACT Attackers are known to utilize domain generation algorithms (DGAs) to generate domain names for command and control (C&C) servers and facilitate the distribution of uniform resource locators within malicious software. DGAs pose a significant threat to cybersecurity owing to their ability to dynamically generate unpredictable domain names. Extensive research is currently underway to detect the domain names created using DGAs. However, the high false positive rates when handling benign domain names in non-English languages pose a challenge. Thus, this study proposes a DGA detection method that effectively embeds non-English domain names to focus on Chinese domain names, which are referred to as domain names composed of Pinyin. The proposed method segments domain names into meaningful subwords for effective vector representation. Consequently, the FastText model learns the context information of the segmented subwords and embeds the domain name. Further, the deep learning-based detection model learns the vectorized domain names and determines whether a particular domain name is DGA-generated. We labeled the Chinese domain names among the benign domain names for our experiment. The experimental results show that the proposed method outperforms existing methods across all performance metrics on the entire test dataset. Notably, the proposed method minimizes the false positive rate, thereby enhancing detection reliability. In addition, it exhibits high performance, achieving a recall of 0.9873 and 0.9886 for Chinese and English domain names, respectively. This demonstrates that the proposed method consistently delivers high performance across various metrics and languages.

INDEX TERMS Botnet, deep learning, domain generation algorithm, embedding, subword segmentation.

I. INTRODUCTION

Domain generation algorithms (DGAs) are primarily embedded within malware to facilitate the dynamic generation of a multitude of domain names [1]. At times, malwares contain domain names that enable bots to connect to command and control servers (C&C). Prior to the advent of DGA, attackers hardcoded domain names within malware [2]; however, cybersecurity experts can easily extract hardcoded domain

names [3]. Therefore, attackers began to use DGAs to render it difficult for experts to detect or block domain names [4]. The DGA communication process involves an attacker, C&C server, and a bot operating as follows. First, the attacker executes a DGA to generate multiple domain names. Second, the attacker registers certain generated domain names and links them to the IP address of the C&C server. Finally, the malware embedded within the bot executes the same DGA to generate domain names, querying them individually. Successful queries facilitate communication between the bot and the C&C server through resolved domain names. DGAs

The associate editor coordinating the review of this manuscript and approving it for publication was Daniel Augusto Ribeiro Chaves¹.

can be used in PCs and IoT devices, drones, or vehicles, posing significant safety and security risks.

Each DGA uses a unique algorithm to generate numerous domain names [5]. The number of potential domain name permutations and the resemblance between DGA-generated and benign domains is dependent on the algorithm [6]. DGAs can be classified into random and dictionary-based DGAs depending on the method of generating the domain names [7]. Random-based DGAs typically employ arbitrary random seeds to generate strings [8]. They facilitate effective evasion of detection. However, random domain names are distinct from benign domain names, which renders them susceptible to detection. In contrast, dictionary-based DGAs create domain names by concatenating sequences of words from one or more word lists [9]. Word lists can be directly included in the malicious binary code or accessed from public sources. Owing to the use of actual words in the creation process, these domain names appear as natural as benign domain names.

DGA-generated domain names should be detected rapidly. This is because DGAs can generate numerous domain names, which can circumvent malware detection systems. Security experts detect DGA-generated domain names using reverse-engineered malware or statistical analysis. However, these techniques are time-consuming and attackers can easily bypass these patterns [10]. Machine- and deep-learning methods have also been used to detect DGA [11], [12], [13]. These methods have garnered attention because of their advantage in quickly identifying DGA domain names without the use of additional contextual information [14].

Existing studies have proposed machine- and deep-learning-based DGA detection methods that exhibit high detection performance. However, these methods are plagued by high false-positive rates owing to non-English-based benign domain names being incorrectly detected as DGA domain names (see Section IV, Table 3 and Table 4). In this paper, non-English domain names are referred to as domain name strings that are Romanized. Relatively high false positive rates have been observed because most deep-learning-based detection models simplistically split the domain names into characters. Consequently, they encode each character numerically. Following encoding, a character-based embedding method maps each encoded character as an independent unit onto a vector. This method can rapidly vectorize domain names with limited preprocessing and capture the relationships between adjacent characters. However, the model encounters difficulties in learning such patterns if specific character combinations or patterns do not appear frequently enough to be detected.

Most non-DGA domain names are based on English and composed by combining words that can be easily remembered by users. Therefore, character-based embedding methods may have limitations in learning non-English domain names, which are relatively rare. In non-English-speaking regions, domain names Romanized from non-English languages constitute a significant proportion of the total. In this

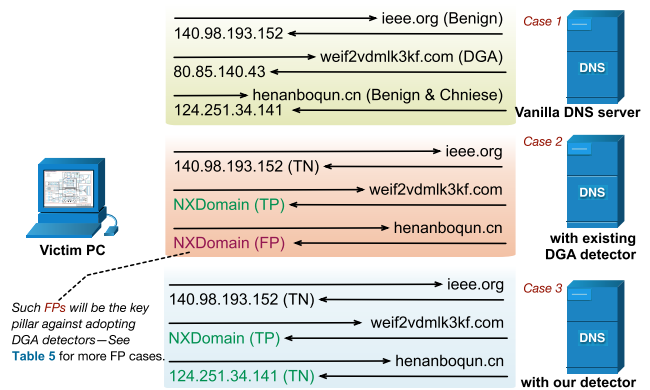


FIGURE 1. Considered problem with existing DGA detectors. In case 1, a DNS server attempts to resolve a DNS query; the victim is exposed to a DGA-based attack. In case 2, a DNS server yields the NXDomain for a DNS query with a possible DGA domain name. Meanwhile, existing detectors could yield false positives. Our method is proposed to this end.

paper, we refer to a *Chinese domain name* as a domain name composed of Pinyin, which is a Romanized form of a Chinese character; for example, “xiaomi.com” and “xinhuanet.com” are considered Chinese domain names. Because Chinese originates from a linguistic family that is completely different than English, the domain names in Pinyin exhibit substantial differences. The character distribution and arrangement of Chinese domain names are distinct from those in English (Figure 3). Consequently, DGA detectors that focus on English domain names are likely to misclassify Chinese domain names as DGA domains. Therefore, it is essential to develop a DGA detection method that can effectively learn non-English domain names.

This study aimed to expand the scope of threat responses by considering domain names based on specific languages, such as Chinese domain names, in DGA detection. We focused on the linguistic characteristics and regional biases in cybersecurity, which are essential elements for understanding the unique security challenges. These challenges are encountered by various users and systems in a global network environment. A limitation of existing methods for detecting DGA domain names is that the domain names using languages with unique features, such as Chinese, may be misclassified as DGA domain names (Figure 1). Consequently, existing DGA detecting methods applied to traffic with a high ratio of Chinese domain names may block legitimate traffic owing to high false positive rates, causing inconvenience to legitimate users. Moreover, failure to carefully examine special cases when protecting security systems or networks can expose them to unexpected threats. Therefore, this study aims to facilitate more comprehensive response to global cyber threats by considering Chinese domain names.

This study proposes a DGA detection method that enhances the classification performance of benign domain names in non-English languages, particularly Chinese. The method also demonstrates high detection performance for

DGA domain names. Chinese was selected considering its significant linguistic differences from English and high global usage ranking. We propose an embedding method that adequately represents various language patterns in domain names, to measure the degree of closeness of subwords within a domain name, thereby segmenting the domain name into meaningful subwords. The FastText word embedding model [15] was trained on the segmented domain names for vectorization. This enabled the learning of the embedding of words based on the context of the subwords constituting the domain name. Thereafter, the embedded domain names were passed through a convolutional neural network and bidirectional long short-term memory network (CNN-BiLSTM)-based detection model to determine whether they are benign or DGA domain names. The contributions of this study can be summarized as follows:

A. UNVEILING MISCLASSIFICATIONS ON CHINESE DOMAIN NAMES

We empirically proved that the existing DGA detection methods are not robust in case of non-English domain names. Specifically, they considerably increase false positives on benign Chinese domain names, which can cause alarm fatigue. Thus, we propose a new embedding method that segments a domain name into meaningful subwords.

B. IMPROVING DETECTION PERFORMANCE THROUGH SUBWORD-BASED EMBEDDING METHOD

We propose a DGA detection method based on a novel embedding approach. The embedding method represents various language patterns effectively. A CNN-BiLSTM model was implemented following the embedding method to accurately determine whether a domain name was benign or DGA. The experimental results confirmed significant remediation of the false positive issues on benign Chinese domain names.

C. ENHANCING MODEL EXPRESSIVENESS THROUGH BI-MODAL FEATURE COMBINATION

We used the bi-modal feature that combines domain name embedding and lexical feature vectors. This approach captured both the semantic differences of words and the structural characteristics in a domain name string. The experimental results confirmed that the combined features maximized the expressiveness and generalization performance of the model.

II. RELATED WORKS

A. MACHINE LEARNING AND DEEP LEARNING-BASED DGA DETECTION METHODS

Recent studies have reported the effectiveness of machine- and deep-learning approaches for DGA detection. Machine learning-based DGA detection primarily focuses on extracting various features from domain names and consequently applying them to machine learning classifiers. These features

include unique characteristics of domain names, such as the proportion of characters composing the domain names [3], [4], [7], [9], [10], [11], [16], [17], n-gram [3], [7], [10], [11], [16], [18] and entropy [16], [7], [9], [17], [3], [10], among others. The algorithms include Random Forest, Logistic Regression, Decision Tree, Support Vector Machine, and Adaboost. For instance, feature-based automated NXDomain classification and intelligence (FANCI) [19] applies a random forest classifier and utilizes the structural, linguistic, and statistical features of domain names, to distinguish between DGA-generated and benign NXDomains.

In contrast, deep learning-based models can eliminate the need for manual feature extraction, primarily using CNNs and LSTMs. CNNs are employed to extract local features from domain names and identify patterns or structures within the domain names. They are used in single or stacked layers, whereby various numbers of filters and kernel sizes are applied to each layer [2], [5], [20], [21], [22], [3], [23]. LSTMs are essential for learning sequence information from domain names and capturing long-term dependencies [24], [8], [25], [26], [3]. The combination of CNNs and LSTMs enhances the detection and classification performance [2], [27], [28], [29]. Some studies have proposed models that combine attention mechanisms with CNNs and LSTMs to improve the detection and classification performance [30], [31], [32], [33], [34], [21], [29]. Further, hybrid models that combine the strengths of machine- and deep-learning approaches [35], [14], [36], [37], [38] have been proposed. Liang et al. [6] applied different detection models of deep neural networks or feature engineering-based models depending on the length of the domain name. This approach addressed the issue of detection performance sensitivity with respect to domain name length characteristics, thereby yielding a model with stable detection capabilities.

B. DOMAIN NAME EMBEDDING APPROACHES

We discuss the existing methods for embedding domain names and detecting DGAs in Table 1. Woodbridge et al. [24] were the first to introduce deep learning for DGA detection. They proposed a significantly compact model comprising embedding, LSTM, and fully connected output layer. This approach is also referred to as the “Endgame.”

Zhang et al. [39] were the first to successfully apply CNN to text classification. They proposed a deep learning model that included multiple stacked CNN layers. Yu et al. [2] discovered a data overfitting issue related to the unique characteristics of domain names when applying this model. Consequently, they experimented with the NYU model, which adjusts the number and size of CNN layers and filters.

Vosoughi et al. [40] proposed an extension of the NYU model comprising multiple CNN and LSTM layers. To address the data-overfitting issue, Yu et al. [2] experimented with a modified architecture that included only one CNN and one LSTM layer, referred to as the MIT model.

A vectorization approach is required to input domain name strings into deep-learning-based models. Therefore,

these methods utilize an embedding technique that separates the domain names at the character level and encodes each character of the domain name into a specific number. Several deep learning detector methods based on character-level embedding have been proposed in addition to the aforementioned three. A characteristic of these methods is that they are more focused on refining the architecture of deep learning-based detectors rather than concentrating on the embedding domain names.

Yun et al. [41] proposed a method for generating adversarial DGA domain names rather than detecting DGAs. They introduced a tokenizer that segmented domain names based on syllables using a bidirectional maximum matching method. Such an attempt to segment domain names was considered novel; however, the segmentation performance decreased significantly for syllables that appeared less frequently in the domain name dataset.

Cucchiarelli et al. [42] proposed a method for vectorizing the domain names represented by n-grams using the Jaccard index and Kullback-Leibler divergence. They tested the detection performance by inputting the embedded domain names into various machine learning algorithms. This was an impressive attempt to represent the similarity of domain names using vectors. Moreover, the method exhibited a higher detection performance than existing deep learning-based methods despite the disadvantage of requiring a long time (an average of 1.8 s required to classify a single domain name).

Xu et al. [43] proposed a method for detecting DGAs via the application of the domain names represented by n-grams to CNNs. They aimed to extract the unique semantic differences between benign and DGA domain names to enhance DGA detection performance.

Morbidoni et al. [26] embedded domain names using unigram, bigram, and trigram. They proposed two deep neural networks that comprised one embedding and LSTM layer each. They trained the Fasttext implementation of Skip-gram and initialized the embedding layer using the trained Fasttext.

Ding et al. [21] proposed the combination of two embedding approaches: character-level (unigram) and bigram-level embedding. This method computed two embedding values from each approach. The authors designed a neural network for classification, which comprised two primary sub-networks. One sub-network was a Transformer-based network utilizing character-level embeddings, whereas the other was a CNN-based network employing bigram-level embeddings.

Huang et al. [22] employed Bidirectional Encoder Representations from Transformers (BERT) for domain name embedding. BERT exploits WordPiece tokenization, and each token can be mapped to a specific vector according to its context. This is significantly different from character-level and n-gram-level embedding. The authors demonstrated the effectiveness of BERT as an embedding layer by training it with domain names and employing it in a CNN network.

The related studies that served as a motivation for this study are discussed. Many studies have proposed methods to detect DGA-generated domains, assuming that benign domain names are based on English. Thus, the datasets used in the experiments and the criteria for embedding domain name strings are centered around English domain names. However, in real-world scenarios, domain names based on languages other than English are prevalent, which renders traditional methods ineffective.

Current deep learning methods primarily utilize character-level embeddings, wherein each character is transformed into a continuous vector. Thus, they may be limited in the case of domain names involving multiple languages and struggle to capture contexts beyond the character level. These models may be adequately trained only with sufficient data. Therefore, further research is required to effectively represent domain name features that extend beyond the character level.

Previous research has primarily focused on the linguistic features that differentiate DGA and benign domain names. Notably, domain names in languages such as Chinese, which have entirely different linguistic features, can be misdetected as random-based DGA domain names.

Recent research on DGA detection aims to effectively detect dictionary-based DGA, which is more challenging than identifying random-based DGA. However, this approach often assumes the use of an English dictionary; consequently, benign domain names containing non-English words can be incorrectly detected as DGA-generated domains. Moreover, attackers may also generate more realistic domain names using Chinese words, which differ from the character pair distribution in English.

In conclusion, effective methods are required for learning various linguistic rules to achieve high detection performance in environments where English and non-English domain names coexist. Thus, this study introduces and evaluates a novel DGA detection method.

III. METHODOLOGY

The proposed method was designed to effectively detect DGA by structurally and semantically segmenting and embedding domain names to derive meaningful domain name representations and consequently train them on a deep learning model.

Figure 2 presents an overview of the entire process of the proposed method. The brief overview of each component is as follows:

- 1) Dataset collection: Benign and DGA domain names are collected for training. Benign domain names are labeled based on whether they are based on Pinyin or not.
- 2) Domain preprocessing: The dataset is refined, and non-alphabetic characters in the domain names are replaced with specific tokens.
- 3) Domain segmentation: Domain names are segmented into meaningful subwords. The segmentation is based on the cohesion probability of subwords. The

TABLE 1. Domain name embedding approaches.

Method	Inclusion of a domain name segmentation approach	Embedding level	Example	Linguistic features	DGA detection model architecture
Endgame [24]	X	Character	g, o, o, g, l, e	X	LSTM
NYU [39]	X	Character	g, o, o, g, l, e	X	CNN
MIT [40]	X	Character	g, o, o, g, l, e	X	CNN-LSTM
Khaos tokenizer [41]	O	Syllable	goo, gle	X	—
DOLPHIN [1]	O	Phonics	g, oo, gl, e	O	RF, XGBoost, SVM
Cucchiarelli <i>et al.</i> [42]	X	N-gram	go, oo, og, gl, le, goo, oog, ogl, gle	X	JL, KL
Xu <i>et al.</i> [43]	X	N-gram	go, oo, og, gl, le, goo, oog, ogl, gle	X	CNN
Morbidoni <i>et al.</i> [26]	X	N-gram	go, oo, og, gl, le, goo, oog, ogl, gle	X	LSTM
Ding <i>et al.</i> [21]	X	N-gram	g, o, o, g, l, e, go, oo, og, gl, le	X	Transformer, CNN
PEPC [22]	X	BERT	go, ##o, ##gle	X	CNN

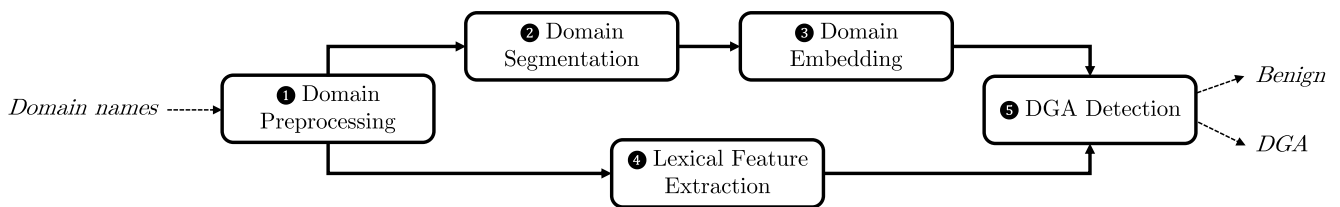


FIGURE 2. Proposed DGA detection method. ① Domain names are preprocessed. First, the second-level domain (SLD) of the domain name is tokenized into its constituent alphabetic and numeric strings, replacing the numeric part with specific tokens. Domain names are vectorized following two methods, through ② and ③ and ④ alone, to effectively represent the subwords and lexical features of domain name strings in the domain name vectors and enhance detection performance. In ② and ③, preprocessed domain names are segmented into n-gram-based subwords. The FastText embedding model then vectorizes the segmented domain names. In ④, lexical features are extracted and vectorized from the preprocessed domain names. In ⑤, a deep learning-based DGA detection model accepts the domain name vectors from ③ and the lexical feature vectors from ④ as input and outputs whether the domain name is DGA-generated.

combination of subwords that maximizes the sum of probabilities is selected.

- 4) Domain embedding: The segmented domains are embedded through the FastText model, which has been trained on the sequences of subwords from the segmented domain names.
- 5) Lexical feature extraction: Lexical features are extracted and vectorized from the domain names before they are replaced with specific tokens.
- 6) DGA detection: A deep learning-based model trained on vectorized domain names detects the DGA domain names. Two inputs are fed into the detection model: the embedded domain name and the lexical feature vector of the domain name.

A. DOMAIN NAME LABELING AND ANALYSIS

1) BENIGN DOMAIN NAME LABELING

Using a single existing language detection library to determine the language of domain names, particularly non-English domain names, was not appropriate for two reasons. First, language detection libraries for natural language processing (NLP) typically target texts written in the characters of specific languages. However, non-English domain names are often written in Romanized characters and use the alphabet regardless of the language. For example, existing Chinese language detection libraries focus on whether the text is composed of Chinese characters. Therefore, these libraries are not suitable for detecting Chinese domain names written in Pinyin. Second, existing language detection libraries

work well with texts that have spaces. However, domain names do not contain spaces. Thus, using existing language detection libraries to detect the language of domain names is challenging.

Therefore, we devised a method for labeling domain names based on Chinese by adopting a language detection algorithm (e.g., googletrans [44]) and word segmentation algorithms (e.g., wordsegment [45], pyenchant [46]). The steps involved in labeling are as follows:

- 1) If the domain name remains unchanged when translated to Chinese using googletrans, it is classified as English.
- 2) This step targets the domain names shorter than three characters, with numbers removed from either the beginning or end of the domain name string. If the top-level domain (TLD) of the domain name does not contain Chinese-speaking country codes, it is classified as English. However, if it has Chinese-speaking code and includes Chinese syllables, it is classified as a Chinese domain name.
- 3) Domain names without Chinese syllables are classified as English.
- 4) If the pronunciation matches the domain name when translated to Chinese using googletrans, it is classified as Chinese.
- 5) Domain names that are determined to comprise only English words, using the Python libraries wordsegment and pyenchant, are classified as English.
- 6) Domain names that cannot be labeled using the above steps are classified as English.

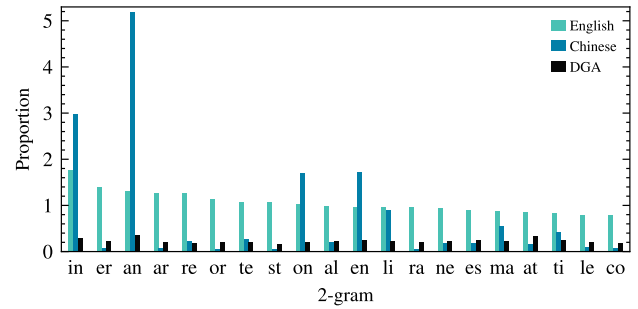
2) DOMAIN NAME ANALYSIS

We analyzed the dataset of domain name strings based on n-grams. This is because the domain names can be regarded as short collections of natural language segments. When creating domain names, owners typically choose names that can be easily remembered. Domain names are typically concise, comprising common spellings and words familiar to users, and selected to be easily pronounceable by users. Thus, they are short collections of natural language segments. The n-gram analysis of the domain names revealed significant differences in the frequency and patterns based on the language characteristics and pronunciation of the domain name compositions. For the analysis, the domain names were segmented into overlapping fragments of ‘n’ consecutive characters. For instance, converting the domain name *google* into bigrams yields [“go,” “oo,” “og,” “gl,” and “le”]. This method provides a clearer understanding of the structure and patterns of domain names.

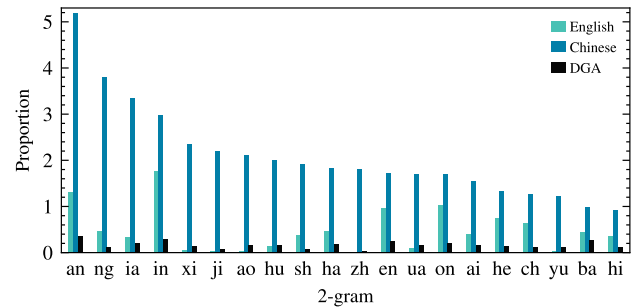
Figure 3 provides the distribution of bigrams. Figure 3a–3c represent the top 20 bigram frequency proportions of benign English, Chinese, and DGA domain names, respectively. Each graph not only shows the bigram frequency of the primary domain name category but also includes the frequency proportions for the other two domain name categories. Figure 3d compares the top 10 bigram frequency proportions for benign English, Chinese, and DGA domain names. Despite the smaller amount of data, Chinese domain names exhibit a distinct bigram frequency ratio because Chinese contains fewer syllables than English. In summary, the bigram analysis revealed differences in bigram frequencies and patterns according to linguistic characteristics and structures. Chinese domain names have different characteristics in n-gram distribution compared to English domain names, which may lead to a risk of misclassification as DGA-generated upon the application of character-level embedding. Therefore, along with unigrams, the consideration of meaningful bigrams, trigrams, and other patterns in representing domain names can better preserve the linguistic and semantic structures of strings. This improvement can enhance the differentiation between benign English, benign Chinese, and DGA domain names.

B. DOMAIN NAME PREPROCESSING

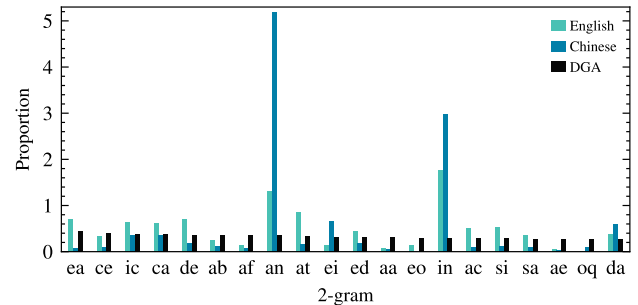
The proposed method focused on the second-level domain (SLD) and removed the TLD, subdomain, duplicated domain names, and domain names containing invalid characters. Each domain name was split into letters and numbers during the preprocessing phase considering distinct characteristics. Alphabetical strings are crucial in terms of their semantic meaning. In contrast, numerical strings are more significant in terms of their length and position. For instance, a domain name such as *example123* is separated into [“example,” “123”]. Consequently, the numerical strings are replaced with special tokens to indicate their numerical nature and



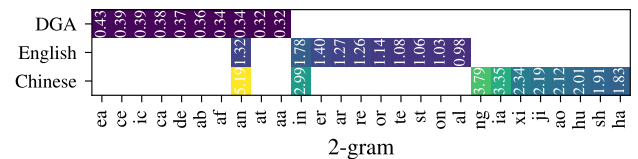
(a) Top 20 2-grams based on English domain names and their proportions in each category.



(b) Top 20 2-grams based on Chinese domain names and their proportions in each category.



(c) Top 20 2-grams based on DGA domain names and their proportions in each category.



(d) Top 10 2-gram populations over three domain categories. Each category is clearly distinguishable. A blank implies not available (N/A) in the top 10 for each category.

FIGURE 3. Top 20 bigram proportions over three domain categories. The x-axis in 3a–3c represents the top 20 bigrams in each category. The three bars for each bigram represent the bigram proportion in each category. Each category can be clearly distinguished by the bigrams with high proportions.

length. For example, *123* would be converted to *<NUM>3*. This approach effectively handles the domain names mixed with letters and numbers, thereby facilitating a clear distinction between numbers and regular words during the embedding process.

C. SUBWORD-BASED DOMAIN NAME SEGMENTATION

The segmentation of domain names involves the division of the domain names into smaller units, specifically, into meaningful subwords. In conventional NLP, text is typically segmented into words based on spaces, and various NLP libraries are available for this purpose. However, the direct application of conventional text segmentation to the domain names is challenging. Domain names lacking spaces often involve a mixture of languages and include non-semantic elements. To address these issues, we segmented the domain names based on subwords.

For subword segmentation, we utilized the concept of cohesion probability [47]. This measures the degree of closeness of substrings to determine how meaningfully a specific string appears across the entire corpus. Strings with higher cohesion probabilities appear more frequently in the corpus, which aids in the estimation of word boundaries. The cohesion probability is calculated as follows:

$$\text{cohesion}(\mathbf{s}) = \left(\prod_{i=1}^{l_s-1} P(\mathbf{s}_{1:i+1} | \mathbf{s}_{1:i}) \right)^{\frac{1}{l_s-1}}, \quad (1)$$

where $\mathbf{s}$ is the given domain name, l_s is the length of the given $\mathbf{s}$, and $\mathbf{s}_{1:i}$ is the substring of $\mathbf{s}$ between indices 1– i . When calculating the cohesion probability, the n -gram strategy is employed because previous studies [10], [18] have demonstrated its effectiveness for domain name classification. The process of segmenting domain names based on the cohesion probability includes the following steps:

- 1) Domain names are divided into n -grams, and the frequency of each n -gram is calculated. In this case, $n = 4$ identifies subwords at the syllable level.
- 2) Based on the frequencies of each n -gram across all domain names, the cohesion probability is calculated.
- 3) Upon segmenting a domain name, the combination of subwords with the maximum sum of cohesion probabilities is selected.

For example, *xiaomi* is segmented into [“xiao,” “mi”], and *skyroom* is segmented into [“sky,” “room”]. This segmentation approach effectively handles various languages and non-semantic elements in domain names.

D. DOMAIN NAME EMBEDDING

Domain name embedding involves the conversion of each tokenized subword of a domain name into a vector form. In contrast to domain names based on English, Chinese domain names have different character distributions and linguistic characteristics, which pose a risk of misclassification as DGA-generated. Therefore, the proposed method embedded domain names using FastText [15], which accurately captured the internal structure and context of subwords. FastText specializes in subword embedding, representing target words by using the surrounding words. It employs two primary approaches for learning the central and surrounding words: Continuous Bag-of-Words and Skip-gram. These approaches predict the central word based on the surrounding

words, or vice versa. FastText processes domain names within a corpus using a sliding window approach and learns the contextual information of the surrounding words for each central word. This approach enables the vectors of related surrounding words to be updated such that they are closer to each other. Using n -grams to split and learn words, FastText can reflect the intrinsic information of characters in the embedding vectors. This ensures that similar words maintain similar vectors even with word variations. Consequently, FastText can effectively reflect the character distribution and linguistic characteristics of domain names in English and Romanized Chinese.

To implement FastText embedding, we used Gensim [48], a Python library. Further, we applied both benign and DGA domain name datasets for training. The Skip-gram approach was utilized in the embedding process by setting the word vector dimension to 300 and considering the five surrounding words. Once trained, the FastText model was applied to the preprocessed dataset to generate vector values for each domain name.

E. LEXICAL FEATURE EXTRACTION

We analyzed the lexical features of the domain names to extract significant characteristics. The domain names generated using random-based DGA methods typically possess lower subword frequencies, rendering the representation challenging. Various lexical features were also considered to distinguish whether a domain name was benign or DGA-generated.

Initially, multiple lexical features, such as the ratio of consonants to vowels, entropy, length of the domain name, and ratio of numbers, were extracted from the domain names and vectorized. These features were normalized using min-max scaling to exert a uniform influence during model training. Subsequently, these normalized feature vectors were fed into the LightGBM model for training. This model extracted feature importance, and we selected the most crucial features according to their importance. Four significant features were selected as follows:

- length of the domain name
- transition frequency between vowels and consonants
- transition frequency between vowels, consonants, and vowels
- transition frequency between English letters and non-English letters

The normalized vector values of the four selected lexical features and vector values of the domain names obtained using FastText were input into the deep learning model.

F. DGA DETECTION MODEL

The proposed method employed a deep learning model based on the CNN-BiLSTM architecture (Figure 4) to classify benign and DGA domain names. Two types of vectors are employed as inputs to the model: FastText embedding and lexical feature vectors.

In the initial stage, local features are extracted from FastText embedding vectors through two convolutional layers. The first convolutional layer has 256 filters with a kernel size of three and a stride of one. The second convolutional layer has 256 filters, a kernel size of six, and a stride of two. Max pooling layers are interspersed between these convolutional layers to extract the maximum scores of each pattern. Subsequently, a BiLSTM layer is applied to learn the temporal dependencies of sequence data. This layer enhances the model performance by learning both forward and backward information in the data. And a Dropout layer with a probability of 0.1 is added to prevent overfitting.

Lexical feature vectors are input into the model and concatenated with the existing output. The concatenated vector is normalized through batch normalization to stabilize the learning process of the model. The final layer in the model is a fully connected layer with a sigmoid activation function. This facilitates the binary classification of the domain names. The Adam optimizer is used during the learning process, with the learning rate set to 0.001. In addition, 100 epochs and EarlyStopping are applied to stop the training when no further improvement in model performance is detected.

IV. EXPERIMENTAL RESULT

A. DATASET

Benign and DGA domain name datasets were collected from open data sources to ensure reproducibility of the experiments. Benign domain name datasets were collected from Alexa Top 1 Million Domains [49] and Majestic Million [50]. Alexa Top 1 Million Domains provides the top one million domain names based on popularity among global Internet users. Majestic Million also offers the top one million popular domain names, and this list is continuously updated.

For benign domain names, several scholars have chosen the Alexa Top 1 Million Domains in their studies. However, according to Zheng et al. [51], the Alexa list may contain malicious domain names at lower rankings and is susceptible to manipulation by attackers. Therefore, we used both the Alexa and Majestic lists for our datasets in the experiments. We selected domain names that appeared in both the data sources for over 150 days during the same six months. Unreliable domain names in popularity lists were filtered, using only frequently appearing domain names.

Two open datasets, DGArchive [52] and University of Murcia domain generation algorithm (UMUDGA) [53] comprise the DGA domain name datasets. DGArchive includes the most well-known DGA families and is continuously updated. Further, UMUDGA is a recently published relatively complete and detailed dataset on DGA botnets.

Several DGA families from two dataset sources were selected for the experiment. The selection criteria were as follows. First, both types of DGA, random- and dictionary-based DGA, were included. Even within the same random-based DGA, the generation method varied slightly, depending on the DGA family. Therefore, random-based DGA families were selected to include various generation methods. Second,

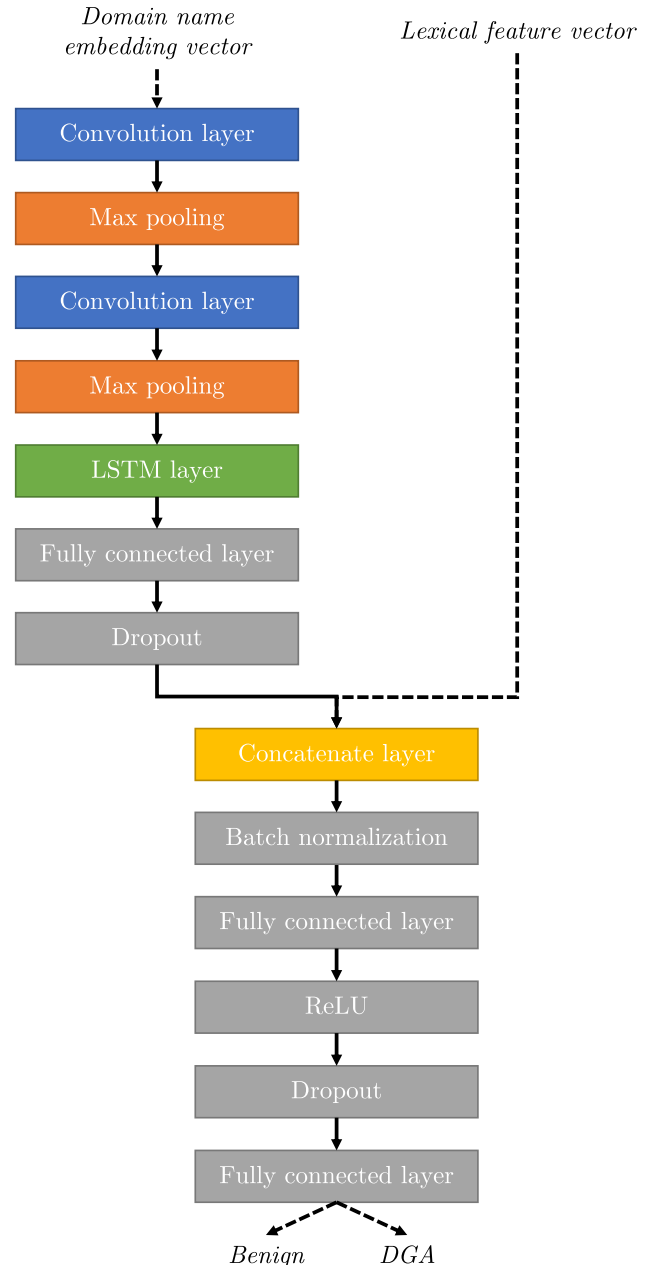


FIGURE 4. Proposed neural network architecture. The convolutional BiLSTM on the left-side extracts features from the embedded domain name, which are concatenated to the particular lexical feature vector from the right-side. The fully connected network then decides whether the domain is DGA-generated. (The dashed lines represent the input and output of the model.).

families with numerous samples and those with fewer samples were chosen. Third, a balanced use of relatively recent and long-existing DGA families was maintained. Finally, DGA families with domain names starting with syllables that only appear in Chinese were included. This was done to ensure the proper classification of benign Pinyin domain names, in the presence of DGA-generated domain names similar to Pinyin. For example, the dataset included SLD names, such as *zhaomen*, *xinuyq*, *xiunfogu*, *xiepuhedo*, *xinatuno*, and *qiuxhufe*.

TABLE 2. Description of the dataset used in the experiment.

Domain label	Dataset source	Category	# names
Benign	Alexa [49], Majestic [50]	Non-Chinese	73,095
		Chinese	1,580
DGA	DGArchive [52], UMUDGA [53]	11 DGA families	100,925
Total			175,600

The selected 11 DGA families were banjori, ccleaner, cryptolocker, dircrypt, dyre, locky, matsnu, pushdo, qadars, ramdo, and symmi. Consequently, 74,675 benign and 100,925 DGA domain names were included (refer to Table 2). The sizes of the benign and DGA datasets were adjusted not to differ significantly.

B. HYPERPARAMETERS

Next, we determined the best hyperparameter setup to maximize the performance of the proposed method. The hyperparameters comprised the number of filters $f_1 \in \{128, 256\}$ and $f_2 \in \{256, 512\}$, kernel size $k_1 \in \{3, 5\}$ and $k_2 \in \{4, 6\}$, number of LSTM units $l \in \{128, 256\}$, number of dense layer units $d_1 \in \{32, 64\}$ and $d_2 \in \{10, 20, 30\}$, dropout rate $r \in \{0.1, 0.25, 0.5\}$, learning rate $lr \in \{0.01, 0.001\}$, and training batch size $b \in \{128, 256, 512\}$. The subscript, *that is* 1 and 2, implies the layer index—*c.f.* the model layout in Figure 4. The goal was to determine the optimal combination of these hyperparameters to maximize the performance of the neural network and minimize false positives. We focused on the metric “Recall (Chinese).”

As the candidate hyperparameters involved 3,456 combinations, we briefly summarize our findings. On average, the proposed neural network yielded an ineffective performance of 0.8830. Meanwhile, the best performance of 0.987342 was obtained with the specific hyperparameters— $f_1 = 256$, $f_2 = 256$, $k_1 = 3$, $k_2 = 6$, $l = 256$, $d_1 = 64$, $d_2 = 30$, $r = 0.1$, $lr = 0.001$, and $b = 128$. Thus, we considered them the optimal hyperparameters and used them for the experiments.

C. COMPARISON WITH EXISTING METHODS

We evaluated the performance of the proposed method through comparisons with representative existing methods. These existing methods included feature-based machine learning models and various architectures of deep learning models. We implemented the existing models and evaluated their detection performance on the same test dataset. Herein, 20% of the collected dataset was used as the test dataset. To compare the performances of the existing and proposed methods, four performance metrics commonly used in machine learning: accuracy, precision, recall, and F1-score, were measured.

- Accuracy represents the ratio of the samples correctly classified by the model.
- Precision indicates the ratio of true positive instances among the instances that the model classified as positive.

TABLE 3. Performance comparison with existing methods. Our method outperforms existing ones.

Method	Accuracy	Precision	Recall	F1-Score	FPR
Endgame [24]	0.9618	0.9707	0.9626	0.9666	0.0392
NYU [39]	0.9637	0.9698	0.9670	0.9684	0.0406
MIT [40]	0.9603	0.9623	0.9689	0.9656	0.0513
B-COS [54]	0.9336	0.9336	0.9336	0.9336	0.0791
B-ResNet [17]	0.9641	0.9641	0.9641	0.9640	0.0465
M-ResNet [17]	0.9723	0.9724	0.9723	0.9723	0.0447
M-ResNet (with TLD) [55]	0.9684	0.9686	0.9684	0.9684	0.0279
EXPLAIN [23]	0.9633	0.9635	0.9633	0.9634	0.0336
FEDCC [29]	0.8724	0.8723	0.8724	0.87243	0.1515
Proposed method	0.9873	0.9915	0.9863	0.9889	0.0114

- Recall signifies the ratio of true positive instances that the model correctly identified among the actual positive instances.
- F1-score is the harmonic mean of precision and recall, representing a balance between the two measurements. It is used to evaluate the overall performance of the model.

Table 3 presents the results of the overall performance comparison with the existing methods. The proposed method exhibited superior performance with accuracy, precision, recall, F1-score, and FPR of 0.9873, 0.9915, 0.9863, 0.9889, and 0.0114, respectively. The proposed method outperformed the other existing methods across all performance metrics, particularly minimizing the FPR to enhance the detection reliability.

Table 4 presents the results of the recall comparison for the Chinese and English domain names. The proposed method exhibited high performance with recall and FPR of 0.9873 and 0.0127, respectively, for Chinese domain names. It also exhibited high performance for English domain names with recall and FPR of 0.9886 and 0.0114, respectively. This was superior to all other methods. The advantage of the proposed method was its consistently high performance for both Chinese and English domain names. Table 4 indicates that the proposed method demonstrated almost no difference in recall between the two languages. In contrast, other methods exhibited significant performance differences depending on the language. B-ResNet [17] and EXPLAIN [23] exhibited the same performance as that of the proposed method for Chinese domain names; however, their performance for English domain names was lacking.

In conclusion, the proposed method demonstrated consistent excellence across various performance metrics and languages. It was particularly effective in detecting non-English domain names, such as Chinese domain names. This demonstrates that the proposed model provides high reliability and efficiency in diverse domain environments.

D. FALSE POSITIVE ANALYSIS

We analyzed the false positive cases for each model in the existing and proposed methods. In particular, we focused on benign Chinese domain names that the proposed method

TABLE 4. Recall comparison results for Chinese/English domain names with existing methods.

Method	Recall (Chinese)	FPR (Chinese)	Recall (English)	FPR (English)
Endgame [24]	0.9114	0.0886	0.9618	0.0382
NYU [39]	0.9114	0.0886	0.9604	0.0396
MIT [40]	0.8987	0.1013	0.9498	0.0502
B-COS [54]	0.9494	0.0506	0.9202	0.0798
B-ResNet [17]	0.9873	0.0127	0.9528	0.0472
M-ResNet [17]	0.9747	0.0253	0.9549	0.0451
M-ResNet (with TLD) [55]	0.8639	0.1361	0.9745	0.0255
EXPLAIN [23]	0.9873	0.0127	0.9658	0.0341
FEDCC [29]	0.5253	0.4747	0.8555	0.1445
Proposed method	0.9873	0.0127	0.9886	0.0114

correctly detected as benign but were classified as malicious by the existing methods.

Table 5 presents the detection results for randomly sampled benign Chinese domain names from the test dataset. Among the nine existing methods, six exhibited false positive cases. These methods struggled with domain names containing subwords frequently used in Chinese but not in English. Such domain names also included numbers or abbreviations, which further complicated the detection process for these methods. The proposed method accurately classified these false-positive cases as benign, thereby demonstrating its ability to distinguish the subword patterns of Chinese domain names from those in DGA domain names.

However, B-ResNet [17], M-ResNet [17], and EXPLAIN [23] correctly detected all the sampled benign Chinese domain names, similar to the proposed method. B-ResNet and EXPLAIN exhibited the same recall as that of the proposed model for Chinese domain names, indicating their effectiveness in the sampled cases. Although M-ResNet exhibited slightly lower recall for Chinese domain names, it did not induce false positives in the sampled cases. This is attributed to the fact that it shares a similar layer structure with B-ResNet. However, as explained in Section IV-C, these methods generally performed worse than the proposed method. They also exhibited performance discrepancies between the Chinese and English domain names.

Consequently, the proposed method effectively reduced the false positives for benign Chinese domain names. This improvement highlighted its superior ability to understand and distinguish the linguistic patterns inherent in Chinese domain names, thereby providing more reliable and accurate detection results. By focusing on these specific cases, we emphasize the robustness and potential of our approach in real-world environments with diverse linguistic patterns.

E. EFFECTIVENESS OF THE BI-MODAL FEATURE

In this section, we evaluate the effectiveness of using the bi-modal features in the proposed method. We used two types of vectors as inputs for the proposed method: domain name embedding and lexical feature vectors. To empirically justify the use of both the vectors, the detection performance differences were compared for the models trained using each

vector individually and when both vectors were used together. Initially, to verify the performance of the domain name embedding vector, we used a model configuration from the proposed detection model architecture, which excluded the Concatenate layer. Subsequently, to derive the performance of the lexical feature vector, only the layers after the Concatenate layer were used in the proposed detection model architecture. The detection performance for each feature vector is shown in Table 6. As evident, the model using the combined bi-modal features (A+B) outperformed those using domain name embedding vectors (A) and lexical feature vectors (B) individually across all performance metrics. Specifically, when using the vectors together, the accuracy was 1.93 and 16.97 percentage points (pp) higher than that when using only the A and B features, respectively. In addition, the model with the bi-modal features exhibited better performance for the both Chinese and English domain names. For the Chinese domain names, the performance difference was notably large when comparing the use of the A feature alone to the bi-modal feature. The use of the bi-modal features yielded an FPR that was 7.27 pp lower. Conversely, for English domain names, the performance difference was significant when comparing the use of the B feature alone to the bi-modal feature. The FPR was 21.77 pp lower when using the A+B features. The enhanced performance was attributed to the combination of diverse information. The embedding vectors captured the semantic differences between words, whereas the lexical feature vectors captured the structural characteristics of the strings. The combination of these vectors enabled the model to leverage richer and more diverse information. Furthermore, the two feature vectors were complementary, thereby providing stronger representational power when used together. Therefore, the bi-modal features effectively enhanced the DGA detection performance in the proposed method.

Furthermore, the bi-modal features mitigated the potential limitations of CNN-BiLSTM. The proposed model employed max pooling layers within the CNN-BiLSTM structure to preserve spatial information while reducing dimensionality. However, pooling operations, including max pooling, can result in the loss of certain spatial information by selecting the maximum value from each pooling window. This can disrupt the continuity of spatial features, potentially impacting the subsequent sequential modeling. Al-Ja'afreh et al. [56] reported that pooling layers in CNN-LSTM tended to discard sensitive information, resulting in lower prediction accuracy. Despite these limitations, the proposed method leveraged bi-modal features to address this issue. The dual representation, which combined domain name embedding and lexical feature vectors, compensated for the spatial information loss caused by max pooling.

F. EFFECTIVENESS OF N-GRAM VALUE

An experiment was conducted to evaluate the effectiveness of the n-gram based subword segmentation in the proposed method. We built corpora based on various n-gram values and

TABLE 5. Randomly sampled benign Chinese domain names and their classification results for various methods. ✓: True negative, ✗: False positive.

Domain name	Endgame [24]	NYU [39]	MIT [40]	B-COS [54]	B-ResNet [17]	M-ResNet [17]	M-ResNet (with TLD) [55]	EXPLAIN [23]	FEDCC [29]	Proposed
59xihuan	✗	✓	✓	✓	✓	✓	✗	✓	✓	✓
99yuhudie	✓	✗	✓	✓	✓	✓	✗	✓	✓	✓
hehuoxinbaiyou	✓	✗	✗	✓	✓	✓	✓	✓	✗	✓
henanboqun	✗	✗	✗	✓	✓	✓	✓	✓	✗	✓
hongqiwanluo	✓	✗	✓	✓	✓	✓	✗	✓	✗	✓
huanjiekj	✓	✗	✓	✓	✓	✓	✗	✓	✗	✓
huxianqiu	✗	✗	✗	✓	✓	✓	✗	✓	✗	✓
jihuoma520	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓
jiuyeqiao	✓	✗	✓	✓	✓	✓	✗	✓	✗	✓
lishaowei	✗	✓	✓	✓	✓	✓	✗	✓	✗	✓
mianfeiku	✗	✗	✗	✓	✓	✓	✗	✓	✗	✓
nbxueyou	✗	✓	✗	✓	✓	✓	✗	✓	✗	✓
qdyilong	✗	✗	✓	✓	✓	✓	✓	✓	✗	✓
xiangfuqu	✗	✓	✓	✗	✓	✓	✗	✓	✗	✓
xinshijie-zhongxin	✗	✗	✓	✗	✓	✓	✗	✓	✓	✓
xmluohu	✗	✗	✗	✓	✓	✓	✗	✓	✗	✓
xuexiluxian	✗	✗	✓	✗	✓	✓	✗	✓	✗	✓
yalianjx	✓	✗	✓	✓	✓	✓	✓	✓	✗	✓
yixuju	✗	✓	✗	✓	✓	✓	✓	✓	✗	✓
yuzhiwei	✗	✗	✓	✓	✓	✓	✗	✓	✗	✓
zghuixin	✓	✗	✓	✓	✓	✓	✓	✓	✗	✓
zhaoshang-gov	✗	✓	✗	✓	✓	✓	✗	✓	✓	✓
zhouliangzhao88	✗	✓	✓	✗	✓	✓	✓	✓	✗	✓
zuyouyizu	✓	✗	✓	✓	✓	✓	✗	✓	✗	✓

TABLE 6. Empirical support for the effectiveness of using bi-modal features: (A: Domain name embedding vector, B: Lexical feature vector) as input of the proposed method.

Feature	Accuracy	Precision	Recall	F1-Score	FPR	Recall (Chinese)	FPR (Chinese)	Recall (English)	FPR (English)
A	0.9680	0.9756	0.9685	0.9721	0.0327	0.9146	0.0854	0.9685	0.0315
B	0.8176	0.8348	0.8510	0.8428	0.2275	0.8449	0.1551	0.7709	0.2291
A+B (proposed)	0.9873	0.9915	0.9863	0.9889	0.0114	0.9873	0.0127	0.9886	0.0114

compared the model's detection performance. Specifically, we set n-gram values of $n \in \{2, 3, 4, 5, 6\}$. For instance, if the n-gram value was set to 3, the corpus was constructed using unigrams, bigrams, and trigrams of the domain names. For each corpus, the domain names were tokenized into subwords and embedded. Consequently, the embedded domain names were used to train the CNN-BiLSTM model, and the performance was evaluated using various metrics.

The detection performance for each n-gram setting is summarized in Table 7. As shown in Table 7, the performance of the model varied across different n-gram values. Specifically, $n = 2$ exhibited the highest overall performance in terms of accuracy, precision, recall, F1-score, and FPR. However, focusing on the Chinese domain names, $n = 4$ exhibited the best performance in terms of recall and FPR. When $n = 4$, the model achieved an recall of 0.9873 for Chinese domain names, the highest among all n-gram values. Further, for the English domain names, it demonstrated a recall of 0.9886. In contrast, $n = 2$ exhibited excellent performance for the English domain names with a recall of 0.9918; however, the performance for the Chinese domain names reduced to a recall of 0.9620. These results suggest that while $n = 2$ was effective for the English domain names, $n = 4$ provided a more balanced performance across both the Chinese and English domain names. The n-gram value of

4 effectively captured the contextual information of Chinese domain names, which significantly reduced misclassifications. Although the differences in accuracy, precision, recall, F1-score, and FPR values were not substantial, the difference in recall for the Chinese domain names was significant. Thus, the performance for the Chinese domain names was more sensitive to the n-gram value setting. In conclusion, selecting $n = 4$ as the n-gram value optimized the model's ability to generalize across different languages, particularly improving the performance metrics for the Chinese domain names. This setting effectively captured the contextual information for each category of domain names, providing the optimal classification performance for the proposed method.

G. EFFECTIVENESS OF USING LEXICAL FEATURES

In the proposed method, we aimed to enhance the detection performance of the model by using vectors based on four lexical features of domain names. This was verified by conducting ablation experiments to evaluate the impact of each lexical feature on the model performance.

Table 8 shows the detection performance for different combinations of lexical features. In Table 1, F1, F2, F3, and F4 represent the four lexical features: length of the domain name, transition frequency between vowels and consonants, transition frequency between vowels, consonants,

TABLE 7. Detection performance with different n-gram values.

N-gram value	Accuracy	Precision	Recall	F1-Score	FPR	Recall (Chinese)	FPR (Chinese)	Recall (English)	FPR (English)
$n = 2$	0.9878	0.9934	0.9854	0.9894	0.0088	0.9620	0.0380	0.9918	0.0082
$n = 3$	0.9870	0.9899	0.9875	0.9887	0.0136	0.9589	0.0411	0.9870	0.0130
$n = 4$	0.9873	0.9915	0.9863	0.9889	0.0114	0.9873	0.0127	0.9886	0.0114
$n = 5$	0.9862	0.9909	0.9850	0.9880	0.0122	0.9620	0.0380	0.9884	0.0116
$n = 6$	0.9840	0.9868	0.9853	0.9860	0.0178	0.9272	0.0728	0.9834	0.0166

TABLE 8. Detection performance with different combinations of lexical features. F1: length of the domain name, F2: transition frequency between vowels and consonants, F3: transition frequency between vowels, consonants, and vowels, F4: transition frequency between English letters and non-English letters. Symbol ✓ means that the corresponding feature was used as input.

F1	F2	F3	F4	Accuracy	Precision	Recall	F1-Score	FPR	Recall (Chinese)	FPR (Chinese)	Recall (English)	FPR (English)
✓				0.9824	0.9864	0.9829	0.9846	0.0183	0.9589	0.0411	0.9822	0.0178
	✓			0.9778	0.9843	0.9769	0.9806	0.0210	0.9304	0.0696	0.9800	0.0200
		✓		0.9727	0.9777	0.9747	0.9762	0.0300	0.9209	0.0791	0.9711	0.0289
			✓	0.9776	0.9849	0.9759	0.9804	0.0202	0.9082	0.0918	0.9814	0.0186
✓	✓			0.9864	0.9902	0.9860	0.9881	0.0132	0.9589	0.0411	0.9874	0.0126
✓		✓		0.9818	0.9874	0.9808	0.9841	0.0169	0.9494	0.0506	0.9838	0.0162
✓			✓	0.9840	0.9883	0.9838	0.9860	0.0157	0.9525	0.0475	0.9850	0.0150
	✓	✓		0.9783	0.9851	0.9771	0.9810	0.0200	0.9335	0.0665	0.9810	0.0190
	✓		✓	0.9829	0.9880	0.9821	0.9850	0.0161	0.9430	0.0570	0.9848	0.0152
		✓	✓	0.9796	0.9864	0.9780	0.9821	0.0183	0.9652	0.0348	0.9821	0.0179
✓	✓	✓		0.9857	0.9897	0.9854	0.9876	0.0139	0.9620	0.0380	0.9867	0.0133
✓	✓		✓	0.9866	0.9915	0.9852	0.9883	0.0114	0.9652	0.0348	0.9891	0.0109
✓		✓	✓	0.9845	0.9887	0.9844	0.9865	0.0153	0.9684	0.0316	0.9851	0.0149
	✓	✓	✓	0.9823	0.9892	0.9798	0.9845	0.0144	0.9399	0.0601	0.9866	0.0134
✓	✓	✓	✓	0.9873	0.9915	0.9863	0.9889	0.0114	0.9873	0.0127	0.9886	0.0114

and vowels, and transition frequency between English and non-English letters. The detection performance of the model was evaluated by removing or combining these features. The results showed that using all lexical features (when all ✓ are checked) yielded the highest detection performance. This result indicated that using all the lexical features in the proposed method was crucial for capturing the structural characteristics of domain names. This was particularly effective in distinguishing the Chinese domain names.

H. EMBEDDING MODEL PERFORMANCE ANALYSIS

The FastText word-embedding method was adopted to embed the domain names. First, the performance was compared with that of other word-embedding method to validate its effectiveness. Subsequently, the impact of the dimensions of the embedding vectors on the detection performance was investigated.

1) COMPARISON WITH OTHER WORD EMBEDDING METHOD

The FastText embedding method was introduced to address sparse data and out-of-vocabulary (OOV) issues. The subwords used in Chinese domain names differ from those typically used in English and are typically less frequent. Therefore, FastText was applied to embed rare or newly emerging subwords accurately. To verify whether FastText embeddings improved the performance of the detection model, we compared its results with those of Word2Vec [57]. Word2Vec is a widely used embedding method in NLP. However, it does not consider the internal structure of words, which implies that it may have limitations in representing

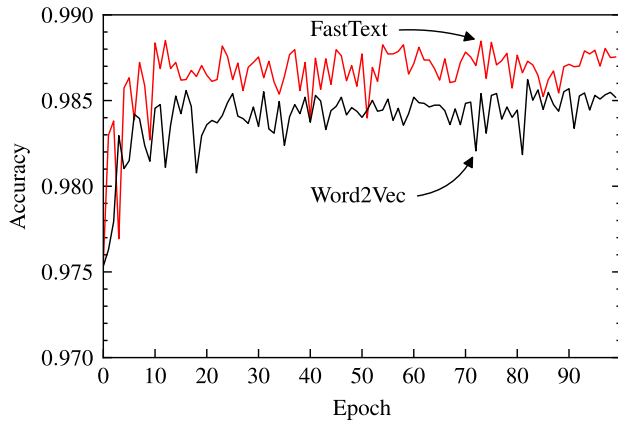
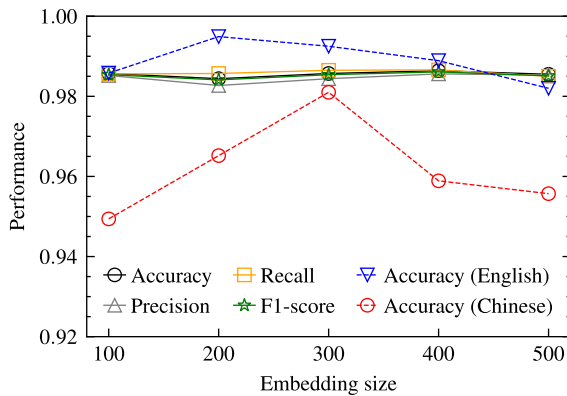
morphologically complex or rare words. We trained the FastText and Word2Vec models on the same training dataset, using the same hyperparameter settings for performance comparison. Subsequently, we used the trained embedding models to generate vectors for the domain names, inputting them into the same detection model to measure accuracy over 100 training epochs. This facilitated the comparison of the learning speed and final accuracy of the model.

Figure 5 exhibits the accuracy changes of the detection models using FastText and Word2Vec over 100 epochs. With Word2Vec, the accuracy increased rapidly in the initial epochs; however, it decreased after the 5th epoch. It eventually converged to a lower accuracy of 0.9845 compared with FastText. In contrast, FastText exhibited a rapid increase in accuracy up to the 6th epoch and converged to a higher accuracy of 0.9873 compared with Word2Vec.

In addition, we applied both the FastText and Word2Vec embedding models to the DGA detection model and compared their performance. The detection performance for each embedding model is summarized in Table 9. As evident, the model using FastText embeddings outperformed that using Word2Vec embeddings across all performance metrics. Furthermore, the FastText model demonstrated better performance in terms of FPR (an FPR of 0.0114 compared to 0.0127 for the Word2Vec model). Specifically, for the Chinese domain names, the FastText model achieved recall and FPR of 0.9873 and 0.0127, respectively, significantly outperforming the Word2Vec model (recall and FPR of 0.9462 and 0.0538, respectively). Thus, the FastText exhibited better generalization performance for the Chinese domain names. In addition, the test dataset embedded with

TABLE 9. Comparison of detection performance between FastText and Word2Vec Embedding Models.

Embedding model	Accuracy	Precision	Recall	F1-Score	FPR	Recall (Chinese)	FPR (Chinese)	Recall (English)	FPR (English)
Word2Vec	0.9845	0.9906	0.9823	0.9864	0.0127	0.9462	0.0538	0.9882	0.0118
FastText	0.9873	0.9915	0.9863	0.9889	0.0114	0.9873	0.0127	0.9886	0.0114

**FIGURE 5.** Comparison results of detection model accuracy over 100 epochs using FastText and Word2Vec. The detection model trained on domain names using FastText outperformed the model using Word2Vec in terms of both training speed and accuracy.**FIGURE 6.** Performance comparison results on the dimensions of embedding vectors. The performance of Chinese domain names exhibited significant variation with dimension, achieving the highest accuracy for 300 dimensions.

the Word2Vec model resulted in 1,578 OOV tokens, which degraded the model performance. In contrast, the FastText model effectively handled OOV words by using subword information, thereby mitigating this issue. Therefore, we conclude that using the FastText embedding model in the proposed method is optimal.

2) PERFORMANCE COMPARISON ACCORDING TO EMBEDDING VECTOR DIMENSIONS

The number of dimensions of an embedding vector is a critical parameter in the embedding process. We compared the performance of the detection model across various dimensions to determine the optimal embedding vector

dimensions. As shown in Figure 6, the accuracy, precision, recall, and F1-score for the entire training set of domain names were ≈ 0.98 , indicating consistent performance across all dimensions. However, the accuracy for the benign Chinese domain names varied, ranging from 0.95–0.98, depending on the dimension. The highest accuracy for the benign Chinese domain names was achieved using 300 dimensions. In contrast, the highest accuracy was observed for 200 dimensions for the benign English domain names, and the performance variation with dimension size was less pronounced than that for the Chinese domain names. Consequently, we set the number of dimensions of the embedding vector to 300.

V. CONCLUSION

This study proposes a method to effectively detect malicious domain names generated by DGAs. Specifically, we aimed to reduce the false-positive rate of Chinese domain names. The proposed method segmented the domain name into meaningful subwords and learnt the vector representations of the domain names. This approach facilitated a more accurate capture of the contextual information of the domain names.

Experimental results confirmed that the proposed method reduced the false positive rate for benign Chinese domain names by an average of $\approx 10\%$. The proposed method effectively enhanced the classification performance for Chinese domain names. Further, its detection performance surpassed that of the existing methods for both Chinese and all domain names. Thus, while our methodology was designed focusing on Chinese domain names, it possesses universality in DGA detection performance. Consequently, our work contributes to expanding the scope of DGA detection and provides a more robust response to malware attacks.

Future research plans include the exploration of methods to further improve the detection performance for domain names in various languages, which is expected to be more effective in defending against DGA cyberattacks.

REFERENCES

- [1] D. Zhao, H. Li, X. Sun, and Y. Tang, "DOLPHIN: Phonics based detection of DGA domain names," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2021, pp. 1–6.
- [2] B. Yu, J. Pan, J. Hu, A. Nascimento, and M. De Cock, "Character level based detection of DGA domain names," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2018, pp. 1–8.
- [3] M. Hassaoui, M. Hanini, and S. E. Kafhali, "Unsupervised clustering for a comparative methodology of machine learning models to detect domain-generated algorithms based on an alphanumeric features analysis," *J. Netw. Syst. Manage.*, vol. 32, no. 1, p. 18, Jan. 2024, doi: [10.1007/s10922-023-09793-6](https://doi.org/10.1007/s10922-023-09793-6).
- [4] T. A. Tuan, N. V. Anh, T. T. Luong, and H. V. Long, "UTL_DGA22—A dataset for DGA botnet detection and classification," *Comput. Netw.*, vol. 221, Feb. 2023, Art. no. 109508.

- [5] S. Zhou, L. Lin, J. Yuan, F. Wang, Z. Ling, and J. Cui, "CNN-based DGA detection with high coverage," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2019, pp. 62–67.
- [6] J. Liang, S. Chen, Z. Wei, S. Zhao, and W. Zhao, "HAGDetector: Heterogeneous DGA domain name detection model," *Comput. Secur.*, vol. 120, Sep. 2022, Art. no. 102803.
- [7] F. Casino, N. Lykousas, I. Homoliak, C. Patsakis, and J. Hernandez-Castro, "Intercepting hail hydra: Real-time detection of algorithmically generated domains," *J. Netw. Comput. Appl.*, vol. 190, Sep. 2021, Art. no. 103135.
- [8] H. Shahzad, A. R. Sattar, and J. Skandaraniyam, "DGA domain detection using deep learning," in *Proc. IEEE 5th Int. Conf. Cryptography, Secur. Privacy (CSP)*, Jan. 2021, pp. 139–143.
- [9] C. Patsakis and F. Casino, "Exploiting statistical and structural features for the detection of domain generation algorithms," *J. Inf. Secur. Appl.*, vol. 58, May 2021, Art. no. 102725.
- [10] J. Selvi, R. J. Rodríguez, and E. Soria-Olivas, "Detection of algorithmically generated malicious domain names using masked N-grams," *Expert Syst. Appl.*, vol. 124, pp. 156–163, Jun. 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417419300648>
- [11] J. Mao, J. Zhang, Z. Tang, and Z. Gu, "DNS anti-attack machine learning model for DGA domain name detection," *Phys. Commun.*, vol. 40, Jun. 2020, Art. no. 101069.
- [12] K. H. Park, H. M. Song, J. D. Yoo, S.-Y. Hong, B. Cho, K. Kim, and H. K. Kim, "Unsupervised malicious domain detection with less labeling effort," *Comput. Secur.*, vol. 116, May 2022, Art. no. 102662.
- [13] H. Lee and H. K. Kim, "Mitigating false positives in DGA detection for non-english domain names," in *Proc. 54th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.-Supplemental Volume (DSN-S)*, Jun. 2024, pp. 150–151.
- [14] R. Sivaguru, J. Peck, F. Olumofin, A. Nascimento, and M. De Cock, "Inline detection of DGA domains using side information," *IEEE Access*, vol. 8, pp. 141910–141922, 2020.
- [15] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Trans. Assoc. Comput. Linguistics*, vol. 5, pp. 135–146, Dec. 2017.
- [16] X. D. Hoang and X. H. Vu, "An improved model for detecting DGA botnets using random forest algorithm," *Inf. Secur. J., Global Perspective*, vol. 31, no. 4, pp. 441–450, Jul. 2022.
- [17] A. Drichel, N. Faerber, and U. Meyer, "First step towards EXPLAINable DGA multiclass classification," in *Proc. 16th Int. Conf. Availability, Rel. Secur.*, Aug. 2021, pp. 1–13.
- [18] H. Zhao, Z. Chang, G. Bao, and X. Zeng, "Malicious domain names detection algorithm based on N-gram," *J. Comput. Netw. Commun.*, vol. 2019, pp. 1–9, Feb. 2019, doi: [10.1155/2019/4612474](https://doi.org/10.1155/2019/4612474).
- [19] S. Schuppen, D. Teubert, P. Herrmann, and U. Meyer, "FANCI: Feature-based automated NXDomain classification and intelligence," in *Proc. USENIX Secur. Symp.*, 2018, pp. 1165–1181.
- [20] D. S. Berman, "DGA CapsNet: ID application of capsule networks to DGA detection," *Information*, vol. 10, no. 5, p. 157, Apr. 2019.
- [21] L. Ding, P. Du, H. Hou, J. Zhang, D. Jin, and S. Ding, "Botnet DGA domain name classification using transformer network with hybrid embedding," *Big Data Res.*, vol. 33, Aug. 2023, Art. no. 100395. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S221457962300028X>
- [22] W. Huang, Y. Zong, S. Shi, L. Wang, and P. Liu, "PEPC: A deep parallel convolutional neural network model with pre-trained embeddings for DGA detection," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2022, pp. 1–8.
- [23] A. Drichel and U. Meyer, "False sense of security: Leveraging XAI to analyze the reasoning and true performance of context-less DGA classifiers," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*. New York, NY, USA: Association for Computing Machinery, Oct. 2023, pp. 330–345, doi: [10.1145/3607199.3607231](https://doi.org/10.1145/3607199.3607231).
- [24] J. Woodbridge, H. S. Anderson, A. Ahuja, and D. Grant, "Predicting domain generation algorithms with long short-term memory networks," 2016, *arXiv:1611.00791*.
- [25] Y. Chen, B. Pang, G. Shao, G. Wen, and X. Chen, "DGA-based botnet detection toward imbalanced multiclass learning," *Tsinghua Sci. Technol.*, vol. 26, no. 4, pp. 387–402, Aug. 2021.
- [26] C. Morbidoni, L. Spalazzi, A. Teti, and A. Cucchiarelli, "Leveraging n-gram neural embeddings to improve deep learning DGA detection," in *Proc. 37th ACM/SIGAPP Symp. Appl. Comput.* New York, NY, USA: Association for Computing Machinery, Apr. 2022, pp. 995–1004, doi: [10.1145/3477314.3507269](https://doi.org/10.1145/3477314.3507269).
- [27] Y. Wang, R. Pan, Z. Wang, and L. Li, "A classification method based on CNN-BiLSTM for difficult detecting DGA domain name," in *Proc. IEEE 13th Int. Conf. Electron. Inf. Emergency Commun. (ICEIEC)*, Jul. 2023, pp. 17–21.
- [28] K. Highnam, D. Puzio, S. Luo, and N. R. Jennings, "Real-time detection of dictionary DGA network traffic using deep learning," *Social Netw. Comput. Sci.*, vol. 2, no. 2, p. 110, Apr. 2021.
- [29] X. Sun and Z. Liu, "Domain generation algorithms detection with feature extraction and domain center construction," *PLoS ONE*, vol. 18, no. 1, Jan. 2023, Art. no. e0279866, doi: [10.1371/journal.pone.0279866](https://doi.org/10.1371/journal.pone.0279866).
- [30] F. Ren, Z. Jiang, X. Wang, and J. Liu, "A DGA domain names detection modeling method based on integrating an attention mechanism and deep neural network," *Cybersecurity*, vol. 3, no. 1, pp. 1–13, Dec. 2020.
- [31] T. A. Tuan, H. V. Long, and D. Taniar, "On detecting and classifying DGA botnets and their families," *Comput. Secur.*, vol. 113, Feb. 2022, Art. no. 102549.
- [32] X. Pei, S. Tian, L. Yu, H. Wang, and Y. Peng, "A two-stream network based on capsule networks and sliced recurrent neural networks for DGA botnet detection," *J. Netw. Syst. Manage.*, vol. 28, no. 4, pp. 1694–1721, Oct. 2020.
- [33] L. Yang, G. Liu, J. Wang, H. Bai, J. Zhai, and Y. Dai, "Fast3DS: A real-time full-convolutional malicious domain name detection system," *J. Inf. Secur. Appl.*, vol. 61, Sep. 2021, Art. no. 102933.
- [34] L. Sidi, Y. Mirsky, A. Nadler, Y. Elovici, and A. Shabtai, "Helix: DGA domain embeddings for tracking and exploring botnets," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2020, pp. 2741–2748.
- [35] P. Karunakaran, "Deep learning approach to DGA classification for effective cyber security," *J. Ubiquitous Comput. Commun. Technol.*, vol. 2, no. 4, pp. 203–213, Jan. 2021.
- [36] C. Zhao, Y. Zhang, and Y. Wang, "A feature ensemble-based approach to malicious domain name identification from valid DNS responses," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–7.
- [37] C. Hwang, H. Kim, H. Lee, and T. Lee, "Effective DGA-domain detection and classification with TextCNN and additional features," *Electronics*, vol. 9, no. 7, p. 1070, Jun. 2020.
- [38] N. T. Cam and N. N. Man, "Uit-DGAdetector: Detect domains generated by algorithms using machine learning," *Cluster Comput.*, Mar. 2024, doi: [10.1007/s10586-024-04363-0](https://doi.org/10.1007/s10586-024-04363-0).
- [39] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1–9.
- [40] S. Vosoughi, P. Vijayaraghavan, and D. Roy, "Tweet2Vec: Learning tweet embeddings using character-level CNN-LSTM encoder-decoder," in *Proc. 39th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2016, pp. 1041–1044.
- [41] X. Yun, J. Huang, Y. Wang, T. Zang, Y. Zhou, and Y. Zhang, "Khaos: An adversarial neural network DGA with high anti-detection ability," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2225–2240, 2020.
- [42] A. Cucchiarelli, C. Morbidoni, L. Spalazzi, and M. Baldi, "Algorithmically generated malicious domain names detection based on n-grams features," *Expert Syst. Appl.*, vol. 170, May 2021, Art. no. 114551.
- [43] C. Xu, J. Shen, and X. Du, "Detection method of domain names generated by DGAs based on semantic representation and deep neural network," *Comput. Secur.*, vol. 85, pp. 77–88, Aug. 2019.
- [44] S. Han. (2020). *Googletrans*. [Online]. Available: <https://pypi.org/project/Googletrans/>
- [45] G. Jenks. (2018). *WordSegment*. [Online]. Available: <https://pypi.org/project/wordsegment/>
- [46] D. Merejkowsky. (2021). *PyEnchant*. [Online]. Available: <https://pypi.org/project/pyenchant/>
- [47] H. Kim. (2019). *Soynlp: Python Package for Korean Natural Language Processing*. [Online]. Available: <https://github.com/lovit/soynlp>
- [48] *Gensim*. Accessed: Jul. 19, 2024. [Online]. Available: <https://pypi.org/project/gensim/>
- [49] (2022). *Alexa Top 1 Million*. [Online]. Available: <https://www.alexa.com/topsites>
- [50] Majestic. (2022). *Majestic Million*. [Online]. Available: <https://majestic.com/reports/majestic-million>
- [51] Y. Zheng, C. Yang, Y. Yang, Q. Ren, Y. Li, and J. Ma, "ShadowDGA: Toward evading DGA detectors with GANs," in *Proc. IEEE ICCCN*, Apr. 2021, pp. 1–8.
- [52] Fraunhofer FKIE. (2022). *DGArchive*. [Online]. Available: <https://dgarchive.caad.fkie.fraunhofer.de/welcome/>

- [53] M. Zago, M. Gil Pérez, and G. Martínez Pérez, "UMUDGA: A dataset for profiling algorithmically generated domain names in botnet detection," *Data Brief*, vol. 30, Jun. 2020, Art. no. 105400.
- [54] M. Böhle, M. Fritz, and B. Schiele, "B-cos networks: Alignment is all we need for interpretability," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 10319–10328.
- [55] A. Drichel, U. Meyer, S. Schüppen, and D. Teubert, "Analyzing the real-world applicability of DGA classifiers," in *Proc. 15th Int. Conf. Availability, Rel. Secur.* New York, NY, USA: Association for Computing Machinery, Aug. 2020, pp. 1–11, doi: [10.1145/3407023.3407030](https://doi.org/10.1145/3407023.3407030).
- [56] M. A. A. Al-Ja'afreh, G. Mokryani, and B. Amjad, "An enhanced CNN-LSTM based multi-stage framework for PV and load short-term forecasting: DSO scenarios," *Energy Rep.*, vol. 10, pp. 1387–1408, Nov. 2023.
- [57] Q. V. Le and T. Mikolov, "Distributed representations of sentences and documents," in *Proc. 31st Int. Conf. Mach. Learn.*, vol. 32, Jan. 2014, pp. 1188–1196.



HUIJU LEE received the B.S. degree in management information systems, convergence security from Korea University, Sejong Campus, Republic of Korea, in 2022, where she is currently pursuing the M.S. degree with the School of Cybersecurity. Her research interests include data-driven security, intrusion detection, and network security.



JEONG DO YOO received the B.S. degree in computer science from Korea University, Seoul, in 2018, where he is currently pursuing the combined M.S. and Ph.D. degree in computer security. His current research interests include computer security, data analytics, and artificial intelligence.



SEONGHOON JEONG (Member, IEEE) received the Ph.D. degree in information security from Korea University, Seoul, Republic of Korea, in 2023. He is currently working as an Assistant Professor with the Division of Artificial Intelligence Engineering, Sookmyung Women's University. His research interests include in-vehicle network security, including intrusion detection systems for controller area networks and automotive Ethernet and data-driven security.



HUY KANG KIM (Member, IEEE) received the Ph.D. degree in industrial and system engineering from Korea Advanced Institute of Science and Technology (KAIST), Republic of Korea, in 2009.

He founded A3 Security Consulting, in 1999 and AI Spera, a data-driven cyber threat intelligence service company, in 2017. He is currently working as a Professor with the School of Cybersecurity, Korea University, Republic of Korea. His recent research interests include

intrusion detection in intelligent transportation systems and in-vehicle networks using machine learning techniques.

...